Task Tracker 專案開發路徑圖：從架構定義到 MVP 實作落地

1. 專案願景與開發策略核心

在區塊鏈應用的生態系中，**Task Tracker** 不僅僅是一個任務管理工具，更是使用者進入鏈上任務互動的關鍵入口。為了確保專案具備高度的擴充性並能快速應對市場回饋，我們採納了**「架構先行、逐步落實」**的開發哲學。這種策略的核心價值在於：優先定義規格而非直接進入編碼 。透過在初期釐清技術邊界與資料型別，我們能有效避免開發中期的架構崩塌，確保每一行程式碼都具備明確的目標。這種嚴謹的預規劃不僅能降低技術債，更為後續整合 Go API 與鏈上合約奠定了穩固的基礎。當規格與型別定義清晰後，專案將進入決定應用基因的首要階段。

2. 階段一：前端規格定義與系統架構規劃

本階段作為專案的藍圖繪製期，旨在確立 MVP（最小可行性產品）的範疇，確保開發資源精準投入於核心價值，並為系統建立穩定的溝通協議。

功能範疇界定

我們採取「核心聚焦、外圍暫緩」的策略，以優化上市時間（TTM）並降低初期整合複雜度：

- **MVP** 核心功能： 包含首頁 Dashboard、錢包連線狀態、任務列表查看、建立新任務、以及單一任務狀態追蹤。
- 暫緩開發功能： AI 驗證機制、NFT/Token 獎勵系統、深度鏈上數據分析、複雜的多錢包切換流程。
- 戰略評價： 暫緩 AI 驗證與獎勵系統可大幅減少初期智慧合約開發與測試的循環週期，讓前端能先在 UI 邏輯上獲得使用者驗證。

技術規格矩陣

身為架構師，優先定義 Primitive Types 是確保資料流穩定的不二法門：| 型別名稱 | 角色說明 | 關鍵欄位定義 || ------ | ------ | ------ || **Task** | 任務核心結構 | id: string, title: string, description: string, status: 'Created' |
| **WalletState** | 錢包狀態管理 | isConnected: boolean, address: string || **TaskSummary** | 首頁數據彙整 | total: number, completed: number, pending: number |

狀態管理決策

在 MVP 階段，我們選擇使用 **React Context API** 來管理錢包與全域狀態，而非導入 Redux 或 MobX。

- 決策理由： Context API 能保持代碼庫輕量，減少樣板代碼（Boilerplate）。
- 前瞻性考慮： 此架構預留了未來遷移至 Wagmi (Web3 專用) 或 Zustand (複雜狀態) 的彈性，確保隨專案規模成長而平滑擴展。當邏輯邊界確立後，下一步將進入開發環境的物理建置，將規範轉化為可執行的環境。

3. 階段二：技術底座建置 — React + TypeScript + Vite 骨架

選擇 **Vite** 搭配 **TypeScript** 是基於現代前端工程對效能與開發體驗（DX）的極致追求。

- 效能評估： Vite 利用原生 ESM 模組實現極速的熱更新（HMR），能讓開發者在調整 UI 組件時獲得秒級回饋，顯著提升開發速率。
- 類型安全： TypeScript 扮演「編譯時守門員」的角色，透過靜態型別檢查在代碼運行前攔截 80% 的邏輯錯誤，降低長期的維護成本。環境就緒後，必須建立標準化的專案結構，以確保團隊在具備共識的語境下進行高度協作。

4. 階段三：標準化目錄結構與基礎檔案配置

結構化的資料夾配置是專案可擴充性的基礎，我們嚴格遵循 權責分離 **(SoC)** 原則進行佈局。

目錄架構視覺化

```
src/
├── components/      # 原子化 UI 元件 (TaskCard, TaskForm,
WalletConnect)
├── context/         # 全域狀態 (WalletContext)
├── hooks/           # 封裝業務邏輯與 API 呼叫
├── layouts/         # 頁面框架 (AppLayout, Header)
├── mocks/           # 靜態模擬資料 (tasks.ts)
├── pages/           # 路由入口頁面 (HomePage, TaskListPage,
TaskDetailPage)
└── types/           # 全域 TypeScript 型別定義定義
```

權責分離 *(SoC)* 評估

將 Mock 資料獨立存放於 src/mocks 具備高度戰略目的。這使得前端頁面在後端 API 或鏈上合約尚未就緒前，即可進行完整的 UI/UX 驗證。未來切換至正式環境時，僅需替換資料獲取層的 Hook 邏輯，無需觸動視覺組件。結構骨架完成後，下一步是定義使用者的導航路徑，建立產品的邏輯動線。

5. 階段四：路由導航與全域版面配置 (Router & Layout)

路由系統構成應用的導引骨架，決定了使用者探索功能的邏輯流。

路由對照表

- **/ (首頁)**：專案入口、願景說明與任務摘要。
- **/tasks (任務列表)**：概覽所有任務狀態。
- **/tasks/:id (任務細節)**：查看單一任務的詳細描述與執行狀態。
- **/tasks/new (建立任務)**：提供任務創建的互動表單。

佈局架構設計

- **AppLayout**：負責維持頁面容器的響應式寬度與主體佈局。
- **Header (Navbar)**：整合 Logo 與導覽列，並將 **Wallet** 區塊 置於此處。這確保了不論使用者位於哪個子頁面，皆能即時監控錢包連線狀態，實現全域一致的連線體驗。路由系統成功引導使用者進入頁面後，重點將轉向核心視覺入口與 UI 組件的開發。

6. 階段五：首頁 UI 組件開發與視覺呈現

首頁是使用者的第一印象，設計策略強調 資訊層級的清晰度 與 行動導向（**Call-to-Action**）。

元件拆解分析

1. **Hero** 區：呈現專案核心價值，提供「查看」與「建立」任務的直覺按鈕。
2. **Wallet** 區：根據連線狀態切換「Connect Wallet」按鈕與「縮寫地址（0x...1234）」顯示。
3. **Summary** 區：透過 TaskSummaryCard 以視覺化卡片呈現任務統計（總數、已完成、待處理）。
4. **Recent Tasks** 區：提取最近三筆任務紀錄，維持首頁的動態資訊更新。

UI/UX 哲學

本專案遵循「 乾淨、白底、卡片式 」的設計方針。作為技術導向的 MVP，我們將其視為一項 設計約束 ：不做過度裝飾與複雜動畫，優先追求「架構清楚、元件可延伸」，以減少開發負擔並降低使用者的認知負荷。視覺框架完成後，最後一步是賦予這些組件動態生命力。

7. 階段六：Mock 資料整合與功能驗證

在正式接軌後端前，透過 Mock 資料進行「模擬驗證」是確保 MVP 完整性的關鍵步驟。

資料流模擬

HomePage 組件將從 src/mocks/tasks.ts 獲取預設數據，並將其注入 TaskSummaryCard 與 TaskCard 中。這不僅驗證了資料傳遞的正確性，也讓利害關係人能預覽真實的使用情境。

MVP 達成檢核清單 *(*目標同步*)*

- x 看到首頁 ：Dashboard 佈局與元件正常渲染。
- x 連接錢包 ：Wallet 狀態區塊能依據 Context 狀態切換顯示。
- x 查看任務列表 ：/tasks 頁面能正確渲染 Mock 數據清單。
- x 建立任務 ：/tasks/new 表單樣式就緒。
- x 查看單一任務狀態 ：/tasks/:id 路由與細節呈現符合規格。
- x 後續可接鏈上交易狀態 ：型別與結構已預留狀態連動介面。本里程碑的達成象徵著前端骨架的完工，為接下來的動態資料對接與鏈上整合掃清了障礙。

8. 專案總結與後續擴充建議

本路徑圖完整勾勒了 Task Tracker 從無到有的建構軌跡，為專案管理者提供了一個可預測、可度量的執行框架。透過「架構先行」的模式，我們在 MVP 階段就確保了程式碼的健壯性。未來擴充建議：

1. **API** 實體替換： 將 src/mocks 切換至真實的 Go Backend API，驗證異步資料獲取與錯誤處理機制。
2. 精細權限管理： 根據錢包連線狀態，加入嚴謹的私人任務訪問權限控制。
3. 鏈上狀態同步： 整合 Web3 Provider，實作任務提交至智慧合約後的交易收據偵測與實時 UI 反饋。此架構為 Task Tracker 奠定了極佳的工程起點，既滿足當前的展示需求，更為未來的去中心化擴展保留了充裕的技術空間。